Q Learning Grid World Project Documentation

Implementation and presentation guide

Project overview

This project implements tabular Q-learning to teach an agent to navigate a fixed grid. The agent starts at S, selects one of four actions, and receives rewards for its movement. Obstacles and grid boundaries block movement. The goal is to reach G with a short route.

The default experiment trains for 3,000 episodes using seed 42. Its greedy policy reaches the goal in 10 moves, matching an independent breadth-first-search shortest-path baseline. The greedy route earns a total reward of 91. All six automated tests pass, including successful shortest-route learning with seeds 0, 7, and 42.

Objectives

- Model states, actions, rewards, and terminal conditions in a grid environment.
- Implement the Q-learning update and epsilon-greedy exploration.
- Save training history and evaluate the learned policy without exploration.
- Validate movement rules and compare the learned route with a shortest-path baseline.

Deliverables

The project includes Python source code, automated tests, a README, example results, this report, a separate GitHub guide, and a 12-slide PowerPoint with speaker notes. Python uses only its standard library. No pip installation is required to run the project.

Report contents: environment model; learning algorithm; code structure; execution guide; experimental results; testing and limitations; GitHub publication.

Environment model

The environment is a deterministic 6 by 6 grid. Each traversable cell is a state identified by a zero-based (row, column) tuple. There are 36 cells: 8 obstacles and 28 traversable cells, including the start and goal. The Q-table stores four values for each of these 28 states.

```
S.....
.###..
...#..
.#....
.#.##.
.....G
```

Symbol or action	Meaning
S	Start at (0, 0)
G	Goal at (5, 5)
#	Blocked cell
.	Traversable cell
0 / 1 / 2 / 3	Up / right / down / left

Transition and reward rules

Event	Reward	Next state
Legal ordinary move	-1	Adjacent open cell
Collision with boundary or obstacle	-5	Current cell
Goal reached	+100	Goal, terminal
Action at the goal	0	Goal remains terminal

A collision does not end the episode. Reaching the goal ends it immediately. Otherwise, training stops the trajectory after 200 actions. The environment validates rectangular maps, accepted symbols, a single start and goal, and reachability before training.

The agent can attempt blocked actions during exploration. Obstacle avoidance means the environment prevents entry and the learned route travels only through valid cells.

Q learning algorithm

Q(s, a) estimates the discounted return from choosing action a in state s and then selecting high-value actions. The trainer initializes every entry to zero. On each transition it moves the selected Q-value toward a target derived from the observed reward.

```
target = reward                          # goal transition
target = reward + gamma * max(Q[next])   # other transitions
Q[state][action] += alpha * (target - Q[state][action])
```

For example, an initial Q-value of 0 updated after a terminal reward of 100 with alpha 0.1 becomes 10. The terminal update does not include any future Q-value.

Parameter	Default	Purpose
alpha	0.1	Learning rate
gamma	0.95	Discount for future reward
episodes	3000	Number of training trajectories
epsilon_start	1.0	Initial random-action probability
epsilon_min	0.05	Minimum exploration probability
epsilon_decay	0.997	Decay factor per episode
max_steps	200	Actions per trajectory
seed	42	Reproducible random choices

Exploration and evaluation

Episode indices start at zero. Epsilon is max(0.05, 1.0 * 0.997 ** episode). With probability epsilon the trainer selects a random action. Otherwise it selects a maximum-Q action, randomly breaking ties with the seeded generator. Q-learning is off-policy because its target uses the best next action even when exploration selects another action.

The evaluator disables exploration and resolves ties in up, right, down, left order. It stops on the goal, a repeated state, or the action limit. Since evaluation is deterministic, a repeated state indicates a cycle. A training time limit truncates the trajectory, so the last update still bootstraps unless the agent reached the goal.

Code structure and saved outputs

File or folder	Responsibility
gridworld.py	Environment, learning, evaluation, visualization, CLI
test_gridworld.py	Six unittest methods
example_map.txt	Default map as editable text
README.md	Quick setup and project explanation
results/	Saved experiment outputs
docs/	Project report and GitHub instructions
presentation/	PowerPoint presentation
.gitignore	Exclude caches and temporary files

Main functions

API	Behavior
GridWorld(rows)	Validate a map and create the state space
GridWorld.step(state, action)	Return next state, reward, and terminal flag
GridWorld.shortest_distance()	Compute the independent BFS path-length baseline
update_q(...)	Apply one terminal or nonterminal Q update
train(env, ...)	Return the learned Q-table and episode history
rollout(env, q, max_steps)	Return greedy path, total reward, and success
render_svg(env, path)	Build a route visualization
main()	Parse CLI options and save experiment files

Output formats

training.csv contains episode, reward, steps, success, and epsilon. q_table.json maps each "row,column" key to values for up, right, down, and left. summary.json records the seed, episode count, greedy result, path, baseline length, and recent training success rate. learned_path.svg displays the grid and route.

The BFS baseline checks reachability and measures the shortest distance. Training and evaluation choose their actions from the learned Q-table. BFS does not construct the learned route.

Installation and execution

Use Python 3.9 or newer. The executed environment used Python 3.11. Open PowerShell in the q_learning_gridworld project folder. Use py on Windows or python where that command identifies your Python installation.

Default experiment

```
py --version
py gridworld.py
py -m unittest -v
```

The default command trains the agent, prints a map with * marking the learned route, prints the JSON summary, and writes results/. Open results/learned_path.svg in a browser. Exit code 0 means greedy evaluation reaches the goal; exit code 1 means it does not.

Custom experiments

```
py gridworld.py --episodes 5000 --seed 7 --max-steps 300
py gridworld.py --map example_map.txt --output-dir custom_results
py gridworld.py --help
```

Custom maps must contain only S, G, #, and . with one start and one reachable goal. Each row must have the same length. Larger or more difficult maps may need more episodes and a larger action limit. Use a different output directory to preserve an earlier experiment.

Python API example

```
from gridworld import GridWorld, train, rollout

env = GridWorld(("S..", ".#.", "..G"))
q, history = train(env, episodes=3000, seed=42)
path, reward, success = rollout(env, q)
print(path, reward, success)
```

Presentation demo sequence

Run the default command, explain the printed route, open the SVG, then run the tests. In PowerPoint Presenter View, use the included notes to explain the Q update and distinguish exploratory training rewards from greedy evaluation.

Experimental results

The saved default experiment uses seed 42, 3,000 episodes, and a 200-action trajectory limit. The final greedy rollout reaches the goal without entering a blocked cell or attempting a collision.

Metric	Observed result
Greedy evaluation success	Yes
Greedy moves	10
Shortest possible moves from BFS	10
Greedy total reward	91
Success in final 100 training episodes	100 of 100

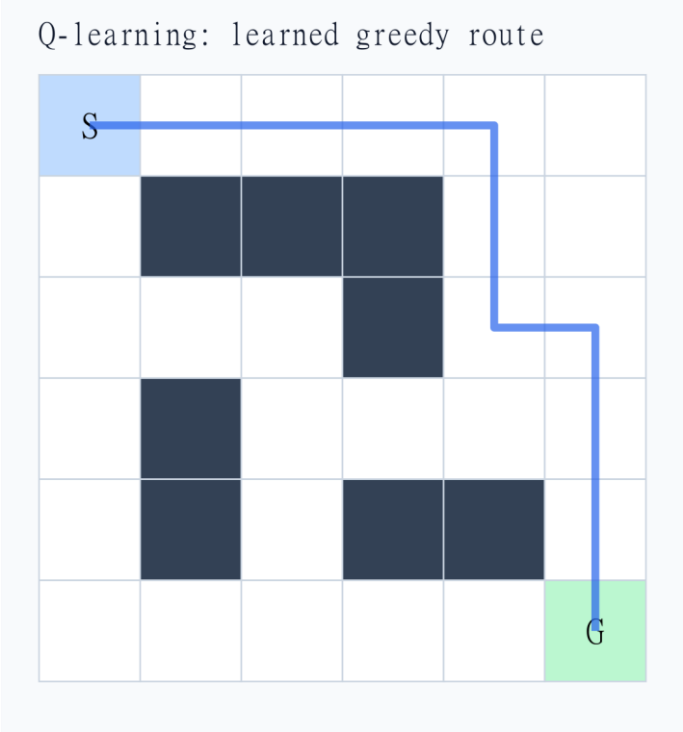


Figure 1 Saved greedy route on the default grid. The blue line connects consecutive learned states. S marks the start and G marks the goal.

The route contains nine ordinary moves and one goal move. Its undiscounted evaluation reward is 9 * (-1) + 100 = 91. Multiple equally short routes can exist. The 100 percent recent training success rate concerns only the final 100 exploratory episodes, not all episodes and not new maps.

The presentation plots mean total reward in consecutive blocks of 100 training episodes from training.csv. These observations include exploration and collisions. The Q update uses discounted future rewards, while the saved episode reward is an undiscounted sum.

Testing and limitations

Test	Evidence checked
test_collision_and_goal	Boundary and wall collisions, goal reward, absorbing goal
test_terminal_update_does_not_bootstrap	Terminal update ignores next-state values
test_nonterminal_update	Discounted target updates the correct value
test_bad_maps	Reject empty, irregular, blocked, duplicate-start, and invalid-symbol maps
test_learning_across_seeds	Seeds 0, 7, 42 reach the goal in 10 valid adjacent moves
test_reproducibility	Identical settings produce identical tables and histories

All six test methods passed in the executed project. The multi-seed test includes three independent training runs on the default grid. This evidence supports the implementation on these settings; it does not prove success for every possible map or seed.

Limitations

The world is small, static, fully observed, and deterministic. The agent learns one table for one map. Changing the layout requires retraining. A fixed finite training budget, fixed learning rate, and the chosen exploration schedule do not provide a general convergence guarantee. The project does not measure performance on unseen maps.

Tabular storage grows with the number of states and actions. The trainer uses O(episodes * max_steps) transitions, with constant work per transition for four actions. Q-table storage is O(states * actions), while saved history grows with the episode count. BFS validation takes O(cells) time and memory for this four-neighbor grid.

Possible extensions

Useful next experiments include larger maps, different reward penalties, stochastic actions, and comparisons with SARSA. A learning-rate schedule and a broader multi-seed evaluation could test stability. Moving obstacles would require a state model that represents their positions.

Publishing the project on GitHub

Use docs/GITHUB_GUIDE.md for the full Windows instructions and troubleshooting. The sequence below assumes Git is installed and an empty repository named q-learning-gridworld exists in your GitHub account. Leave the online README, license, and .gitignore initialization options unchecked.

Local preparation

Open PowerShell inside the project folder that contains gridworld.py and README.md. Confirm the demo and tests work before committing. Replace the name, email, and username placeholders below with your details.

```
git init
git branch -M main
git config user.name "YOUR NAME"
git config user.email "YOUR GITHUB EMAIL"
git status
git add .
git diff --cached --stat
git commit -m "Add Q-learning grid world project"
```

Connection and upload

```
git remote add origin https://github.com/YOUR_USERNAME/q-learning-gridworld.git
git remote -v
git push -u origin main
```

Complete Git Credential Manager browser authentication if prompted. GitHub account passwords do not authenticate HTTPS Git pushes. If your setup requests a token, use a personal access token with repository access. Refresh the GitHub repository page and confirm the files appear.

Future updates

```
git add .
git commit -m "Describe the change"
git push
```

References

Implementation and experiment evidence: gridworld.py, test_gridworld.py, results/summary.json, and results/training.csv in this project.

GitHub Docs, Adding locally hosted code to GitHub: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

GitHub Docs, About remote repositories: https://docs.github.com/en/get-started/git-basics/about-remote-repositories